In [1]:

```python
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
import warnings
import os

warnings.filterwarnings('ignore')

# ===================== 1. 基础设置与路径配置 =====================
# 随机种子保证结果可复现
torch.manual_seed(42)
np.random.seed(42)
# 设备选择（GPU优先）
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"使用设备: {device}")

# 你的目标路径（已确认）
target_dir = r"D:\python 神经网络\1 水泥测试\Concrete_Data_Yeh"
# 自动创建目录（防止路径不存在报错）
if not os.path.exists(target_dir):
    os.makedirs(target_dir)
    print(f"\n已自动创建目录: {target_dir}")

# 定义各文件完整路径
data_path = os.path.join(target_dir, "D:/python 神经网络/水泥测试/Concrete_Data_
corr_plot_path = os.path.join(target_dir, "correlation_analysis.png")  # 相关性
perf_plot_path = os.path.join(target_dir, "model_performance.png")  # 性能图
model_save_path = os.path.join(target_dir, "concrete_strength_model.pth")  # 模

# ===================== 2. 数据加载与列名适配（精准匹配你的列名） ==============
# 加载数据
try:
    df = pd.read_csv(data_path)
    print(f"\n✅ 成功加载数据集: {data_path}")
    print(f"\n你的数据集原始列名: \n{df.columns.tolist()}")
except FileNotFoundError:
    print(f"\n❌ 错误: 未找到文件 {data_path}")
    print("请确认Concrete_Data_Yeh.csv已放在该目录下！")
    exit()

# 精准映射你的列名（核心修复: 完全匹配你提供的列名）
column_mapping = {
    'cement': 'Cement',
    'slag': 'Blast_Furnace_Slag',
    'flyash': 'Fly_Ash',
    'water': 'Water',
    'superplasticizer': 'Superplasticizer',
    'coarseaggregate': 'Coarse_Aggregate',
    'fineaggregate': 'Fine_Aggregate',
    'age': 'Age',
    'csMPa': 'Strength'  # 输出列csMPa映射为统一的Strength
}
```

```python
# 重命名列（只映射存在的列，避免报错）
df.rename(columns={k: v for k, v in column_mapping.items() if k in df.columns},
print(f"\n✅ 列名统一后：\n{df.columns.tolist()}")

# 缺失值处理
print("\n=== 缺失值统计 ===")
missing_stats = df.isnull().sum()
print(missing_stats)
df = df.dropna()  # 删除缺失值（若需填充可改为df.fillna(df.mean(), inplace=True)

# ===================== 3. 相关性分析与特征筛选 =====================
# 计算特征与强度（Strength）的相关性
correlation = df.corr()['Strength'].sort_values(ascending=False)
print("\n=== 各特征与水泥强度的相关性（降序） ===")
print(correlation)

# 可视化相关性（保存到目标目录）
plt.figure(figsize=(10, 6))
ax1 = plt.gca()
correlation.drop('Strength').plot(kind='bar', color='#1f77b4', ax=ax1)
ax1.set_title('Correlation between Features and Concrete Strength', fontsize=14,
ax1.set_xlabel('Features', fontsize=12, labelpad=10)
ax1.set_ylabel('Pearson Correlation Coefficient', fontsize=12, labelpad=10)
ax1.axhline(y=0, color='red', linestyle='--', alpha=0.5)
ax1.tick_params(axis='x', rotation=45, labelsize=10)
ax1.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(corr_plot_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n📊 相关性分析图已保存至：{corr_plot_path}")

# 筛选高相关特征（阈值0.1，可调整）
threshold = 0.2
selected_features = correlation[abs(correlation) > threshold].index.tolist()
selected_features.remove('Strength')  # 移除目标变量
print(f"\n=== 筛选后的高相关特征（相关系数绝对值>{threshold}）===")
print(selected_features)

# 提取特征和目标变量
X = df[selected_features].values  # 筛选后的特征
y = df['Strength'].values.reshape(-1, 1)  # 目标变量（强度）

# ===================== 4. 数据集划分（前80%训练，后20%测试） ==================
train_size = int(0.8 * len(X))
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
print(f"\n=== 数据集划分 ===")
print(f"训练集数量：{len(X_train)} 条")
print(f"测试集数量：{len(X_test)} 条")

# 特征标准化（仅用训练集拟合，避免数据泄露）
scaler_X = StandardScaler()
scaler_y = StandardScaler()  # 目标变量也标准化，提升训练稳定性

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)
y_train_scaled = scaler_y.fit_transform(y_train)
y_test_scaled = scaler_y.transform(y_test)

# 转换为PyTorch张量并移至指定设备
```

```python
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32).to(device)


# ===================== 5. 构建神经网络模型 =====================
class ConcreteStrengthNN(nn.Module):
    def __init__(self, input_dim):
        super(ConcreteStrengthNN, self).__init__()
        # 线性回归本质的神经网络（含隐藏层提升拟合能力）
        self.model = nn.Sequential(
            nn.Linear(input_dim, 64),  # 输入层→隐藏层1
            nn.ReLU(),  # 激活函数（可替换为Sigmoid/Tanh测试）
            nn.Linear(64, 32),  # 隐藏层1→隐藏层2
            nn.ReLU(),
            nn.Linear(32, 1)  # 隐藏层2→输出层（回归任务输出1个值）
        )

    def forward(self, x):
        return self.model(x)


# 初始化模型（输入维度=筛选后的特征数量）
input_dim = len(selected_features)
model = ConcreteStrengthNN(input_dim).to(device)
print("\n=== 神经网络模型结构 ===")
print(model)

# ===================== 6. 模型训练 =====================
# 定义损失函数和优化器
criterion = nn.MSELoss()  # 回归任务用均方误差损失
optimizer = optim.Adam(model.parameters(), lr=0.001)  # Adam优化器

# 训练参数
epochs = 200
train_losses = []  # 记录训练损失
test_losses = []  # 记录测试损失

print("\n=== 开始模型训练 ===")
for epoch in range(epochs):
    # 训练模式（启用梯度计算）
    model.train()
    optimizer.zero_grad()  # 清空梯度

    # 前向传播
    y_pred_train = model(X_train_tensor)
    loss_train = criterion(y_pred_train, y_train_tensor)

    # 反向传播+参数更新
    loss_train.backward()
    optimizer.step()

    # 测试模式（禁用梯度计算）
    model.eval()
    with torch.no_grad():
        y_pred_test = model(X_test_tensor)
        loss_test = criterion(y_pred_test, y_test_tensor)

    # 记录损失
```

```
    train_losses.append(loss_train.item())
    test_losses.append(loss_test.item())

    # 每20轮打印进度
    if (epoch + 1) % 20 == 0:
        print(f"Epoch [{epoch + 1}/{epochs}] | 训练损失: {loss_train.item():.4f}

# ===================== 7. 模型评估 =====================
# 预测并反标准化（还原为MPa单位）
model.eval()
with torch.no_grad():
    y_pred_test_scaled = model(X_test_tensor)
    y_pred_test = scaler_y.inverse_transform(y_pred_test_scaled.cpu().numpy())
    y_test_original = scaler_y.inverse_transform(y_test_scaled)  # 真实值反标准化

# 计算评估指标
mse = mean_squared_error(y_test_original, y_pred_test)
rmse = np.sqrt(mse)  # 均方根误差（更直观，单位MPa）
print(f"\n=== 模型评估结果 ===")
print(f"测试集均方误差（MSE）: {mse:.4f}")
print(f"测试集均方根误差（RMSE）: {rmse:.4f} MPa")

# ===================== 8. 结果可视化 =====================
fig, (ax2, ax3) = plt.subplots(1, 2, figsize=(14, 6))

# 子图1：训练/测试损失曲线
ax2.plot(range(1, epochs+1), train_losses, label='Training Loss', color='#2ca02c
ax2.plot(range(1, epochs+1), test_losses, label='Test Loss', color='#d62728', li
ax2.set_title('Training & Test Loss Curve', fontsize=14, pad=20)
ax2.set_xlabel('Epochs', fontsize=12, labelpad=10)
ax2.set_ylabel('Mean Squared Error (MSE)', fontsize=12, labelpad=10)
ax2.legend(fontsize=10)
ax2.grid(alpha=0.3)
ax2.tick_params(labelsize=10)

# 子图2：真实值vs预测值
ax3.scatter(y_test_original, y_pred_test, alpha=0.6, color='#1f77b4', s=30)
min_str = min(y_test_original.min(), y_pred_test.min())
max_str = max(y_test_original.max(), y_pred_test.max())
ax3.plot([min_str, max_str], [min_str, max_str], 'r--', label='Perfect Predictio
ax3.set_title('True Strength vs Predicted Strength', fontsize=14, pad=20)
ax3.set_xlabel('True Strength (MPa)', fontsize=12, labelpad=10)
ax3.set_ylabel('Predicted Strength (MPa)', fontsize=12, labelpad=10)
ax3.legend(fontsize=10)
ax3.grid(alpha=0.3)
ax3.tick_params(labelsize=10)

plt.tight_layout()
plt.savefig(perf_plot_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n📊 模型性能可视化图已保存至：{perf_plot_path}")

# 保存模型
torch.save(model.state_dict(), model_save_path)
print(f"💾 训练完成的模型已保存至：{model_save_path}")
print("\n🎉 程序运行完成！图片标题正常显示（无方框）。")
```

```
使用设备: cpu

✅ 成功加载数据集: D:/python 神经网络/水泥测试/Concrete_Data_Yeh.csv

你的数据集原始列名:
['cement', 'slag', 'flyash', 'water', 'superplasticizer', 'coarseaggregate', 'fin
eaggregate', 'age', 'csMPa']

✅ 列名统一后:
['Cement', 'Blast_Furnace_Slag', 'Fly_Ash', 'Water', 'Superplasticizer', 'Coarse_
Aggregate', 'Fine_Aggregate', 'Age', 'Strength']

=== 缺失值统计 ===
Cement                0
Blast_Furnace_Slag    0
Fly_Ash               0
Water                 0
Superplasticizer      0
Coarse_Aggregate      0
Fine_Aggregate        0
Age                   0
Strength              0
dtype: int64

=== 各特征与水泥强度的相关性（降序） ===
Strength              1.000000
Cement                0.497832
Superplasticizer      0.366079
Age                   0.328873
Blast_Furnace_Slag    0.134829
Fly_Ash              -0.105755
Coarse_Aggregate     -0.164935
Fine_Aggregate       -0.167241
Water                -0.289633
Name: Strength, dtype: float64

📊 相关性分析图已保存至: D:\python 神经网络\1 水泥测试\Concrete_Data_Yeh\correlat
ion_analysis.png

=== 筛选后的高相关特征（相关系数绝对值>0.2）===
['Cement', 'Superplasticizer', 'Age', 'Water']

=== 数据集划分 ===
训练集数量: 824 条
测试集数量: 206 条

=== 神经网络模型结构 ===
ConcreteStrengthNN(
  (model): Sequential(
    (0): Linear(in_features=4, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=1, bias=True)
  )
)

=== 开始模型训练 ===
Epoch [20/200] | 训练损失: 0.6658 | 测试损失: 0.4801
Epoch [40/200] | 训练损失: 0.4333 | 测试损失: 0.4419
```

```
Epoch [60/200] | 训练损失: 0.3500 | 测试损失: 0.4249
Epoch [80/200] | 训练损失: 0.2879 | 测试损失: 0.4233
Epoch [100/200] | 训练损失: 0.2471 | 测试损失: 0.3884
Epoch [120/200] | 训练损失: 0.2189 | 测试损失: 0.3583
Epoch [140/200] | 训练损失: 0.2003 | 测试损失: 0.3417
Epoch [160/200] | 训练损失: 0.1866 | 测试损失: 0.3572
Epoch [180/200] | 训练损失: 0.1752 | 测试损失: 0.3848
Epoch [200/200] | 训练损失: 0.1664 | 测试损失: 0.4059

=== 模型评估结果 ===
测试集均方误差（MSE）: 124.8666
测试集均方根误差（RMSE）: 11.1744 MPa

📊 模型性能可视化图已保存至：D:\python 神经网络\1 水泥测试\Concrete_Data_Yeh\model_performance.png
💾 训练完成的模型已保存至：D:\python 神经网络\1 水泥测试\Concrete_Data_Yeh\concrete_strength_model.pth

🎉 程序运行完成！图片标题正常显示（无方框）。
```

In [ ]: